

Ankiflix — Master Build Instructions for Gemini Agent (Antigravity IDE)

This document defines the exact architecture, operational constraints, deployment strategy, design philosophy, automation flow, and phased implementation plan for building Ankiflix. Ankiflix is NOT an Anki replacement platform. It is a metadata-first discovery and community platform for educational decks. Primary Goal: Create a premium, Netflix-inspired discovery experience for exam preparation decks while maintaining extremely low operational complexity and near-autonomous long-term operation.

Core Product Definition

- Metadata-first educational deck discovery platform.
- Search, discovery, ranking, reviews, metadata, and community discussions.
- External deck hosting only — Ankiflix does NOT host large .apkg files.
- Users discover decks and are redirected to official/external sources.

Critical Constraints

- DO NOT build native mobile apps initially.
- DO NOT use WordPress or Elementor.
- DO NOT build custom Anki syncing infrastructure.
- DO NOT build AI tutors or chatbots.
- DO NOT overengineer architecture.
- DO NOT use heavy Three.js scenes everywhere.
- DO NOT add unnecessary backend complexity.

Recommended Stack

- Frontend: Next.js 15 + TypeScript
- Styling: Tailwind CSS
- UI Components: shadcn/ui + Radix UI
- Animations: Framer Motion only
- Backend: Supabase Free Tier
- Database: PostgreSQL
- Hosting: Vercel Free Tier
- Automation: GitHub Actions
- Scraper: Python + BeautifulSoup
- Icons: Lucide Icons

Domain & Deployment

- Use subdomain: ankiflix.trochlea.online

- Use Hostverge ONLY for DNS/domain management.
- Deploy frontend using Vercel.
- Connect GitHub repository to Vercel for automatic deployments.
- Every GitHub push should auto-deploy.

Design Philosophy

- Premium does NOT mean overloaded.
- Focus on speed, typography, spacing, consistency, and smoothness.
- Netflix-inspired browsing psychology, not literal cloning.
- Dark-mode cinematic UI.
- Mobile-first responsive experience.
- Fast loading is mandatory.

UI Requirements

- Hero section with trending deck.
- Horizontal scrolling carousels.
- Hover scaling and smooth transitions.
- Deck detail modal/page.
- Subtle premium animations only.
- Minimal GPU-heavy effects.
- Skeleton loading states.
- Large typography and clean spacing.

Core MVP Features

- Homepage hero section.
- Trending categories.
- Deck search.
- Exam filters.
- Deck detail pages.
- Ratings and reviews.
- Comments section.
- External download button.
- SEO metadata generation.
- Responsive mobile design.

Database Schema

- Tables required: decks, reviews, comments, tags, deck_tags.
- Deck metadata should include: title, description, slug, exam category, specialty, card count, download count, last updated, source URL, thumbnail URL, difficulty, review burden.

Automation Requirements

- Python scraper runs daily via GitHub Actions.
- Scrape metadata only.
- Respect external website rate limits.
- Automatically update rankings and trending sections.
- Automatically detect dead links if possible.

Search Requirements

- Use PostgreSQL full-text search initially.
- Prioritize fast filtering.
- Future semantic search optional but not required initially.

SEO Requirements

- Every deck page must generate unique metadata.
- Use clean slugs.
- Generate OpenGraph previews.
- Implement schema markup.
- Optimize for search terms like: best NEET PG deck, best pathology Anki deck, concise surgery deck.

Community Requirements

- Allow lightweight comments and reviews.
- Enable deck-specific discussions.
- Keep moderation lightweight initially.
- Add report system later if needed.

Long-Term Maintenance Philosophy

- Prefer serverless architecture.
- Prefer static generation wherever possible.
- Minimize dependencies.
- Use stable libraries only.
- Avoid unnecessary microservices.
- Keep operational burden low.

Memory / Checkpoint System

- Gemini Agent should maintain project memory checkpoints.
- Create milestone snapshots at each completed phase.
- Use GitHub commits as checkpoint anchors.

- Document architectural decisions in markdown files.
- Use reusable prompt templates and structured task memory.
- Prefer deterministic workflows over agent improvisation.

Suggested Memory Workflow

- PHASE_01_FOUNDATION_COMPLETE
- PHASE_02_UI_SCAFFOLD_COMPLETE
- PHASE_03_DATABASE_CONNECTED
- PHASE_04_SEARCH_COMPLETE
- PHASE_05_DEPLOYMENT_COMPLETE
- PHASE_06_AUTOMATION_COMPLETE

Recommended Repository Structure

- /app
- /components
- /lib
- /styles
- /scripts
- /supabase
- /public
- /docs
- /github/workflows

Phase-wise Execution Plan

- Phase 1: Create Supabase project and initialize Next.js app.
- Phase 2: Configure Tailwind, shadcn/ui, and dark theme.
- Phase 3: Build homepage and category carousels.
- Phase 4: Implement deck detail pages and search.
- Phase 5: Connect Supabase database.
- Phase 6: Create scraper and GitHub automation.
- Phase 7: Configure SEO and deployment.
- Phase 8: Connect Vercel and domain.

Critical Final Instructions

- Maintain simplicity.
- Maintain performance.
- Maintain SEO quality.
- Maintain mobile responsiveness.
- Avoid unnecessary dependencies.

- Prioritize maintainability over complexity.

Master Prompt for Gemini Agent

PROJECT: Ankiflix

Build a production-quality metadata-first Anki deck discovery platform.

PRIMARY GOALS:

- Premium Netflix-inspired browsing UX
- SEO-first architecture
- Lightweight infrastructure
- Autonomous operation
- Low maintenance
- Mobile-first responsiveness

TECH STACK:

- Next.js 15
- TypeScript
- Tailwind CSS
- shadcn/ui
- Framer Motion
- Supabase
- Vercel
- GitHub Actions

IMPORTANT:

- Do NOT host .apkg files
- Use external source links only
- Avoid overengineering
- Avoid unnecessary AI systems
- Avoid heavy animations

DEPLOYMENT:

- Use ankiflix.trochlea.online
- Deploy via Vercel
- Use GitHub auto deployments

AUTOMATION:

- Daily scraper via GitHub Actions
- Metadata extraction only
- Maintain low operational burden

PRIORITY:

Maintainability > Complexity
Performance > Visual overload
SEO > Fancy effects